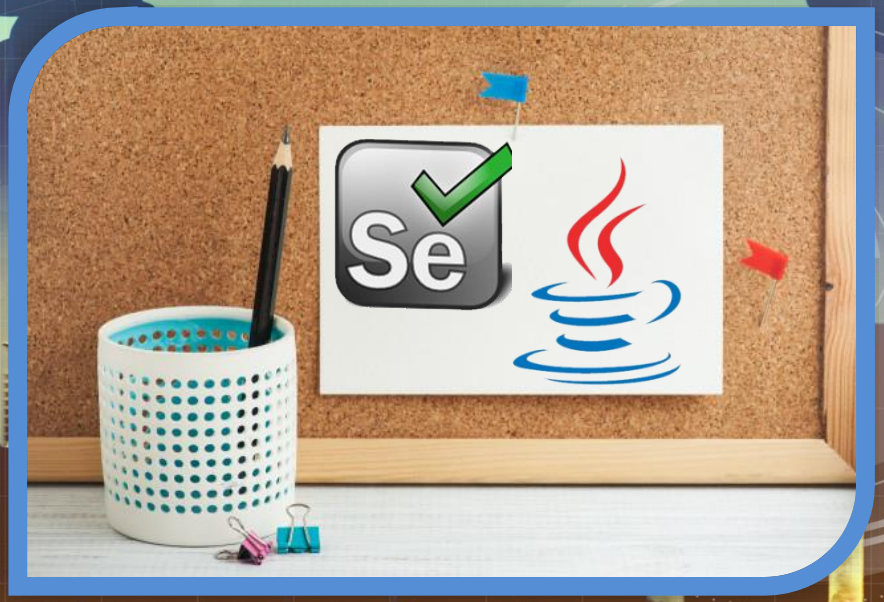
CSS And XPath Techniques WebDriver - Lecture - 3



Agenda

- Locate Elements by Link Text
- CSS Locators
- Creating CSS with Element's Attribute(s)
- More Ways with CSS
  - Class and ID Attribute in CSS
  - CSS For Element containing value
- Completing Login-Logout script
  - Using getTitle() method
  - Using getText() method
- Compound Class Name Error
- Xpath Locators
- More XPath Locating Techniques
  - XPath with Index
  - OR & AND in XPath
  - Contains Value in Xpath
  - Element Text in XPath
- Single Slash(/) And Double Slash(//) in Xpath
- CSS Vs. XPATH
- Quiz
- Homework

Locate Elements by Link Text

- For **anchor <a> tag**, **By class** has two **special locator methods** which comes handy only for locating **hyperlink**:
  - **public static By linkText(String linkText)**
  - **public static By partialLinkText(String partialLinkText)**

- The **<a> tag** defines a **hyperlink**, which is used to link from one page to another and can be accessed using an **exact or partial match of their link text**.
  Suppose, there is a **HTML** snippet:
  <a href=" https://www.qamentor.com/qa-university/ ">Visit QAMentor University</a>
  - Corresponding code for locating WebElement by linktext(String value) method:
    By by1 = By.linkText("Visit QA University");
  - Locating WebElement by partialLinkText(String value) method:
    By by2 = By.partialLinkText("University");

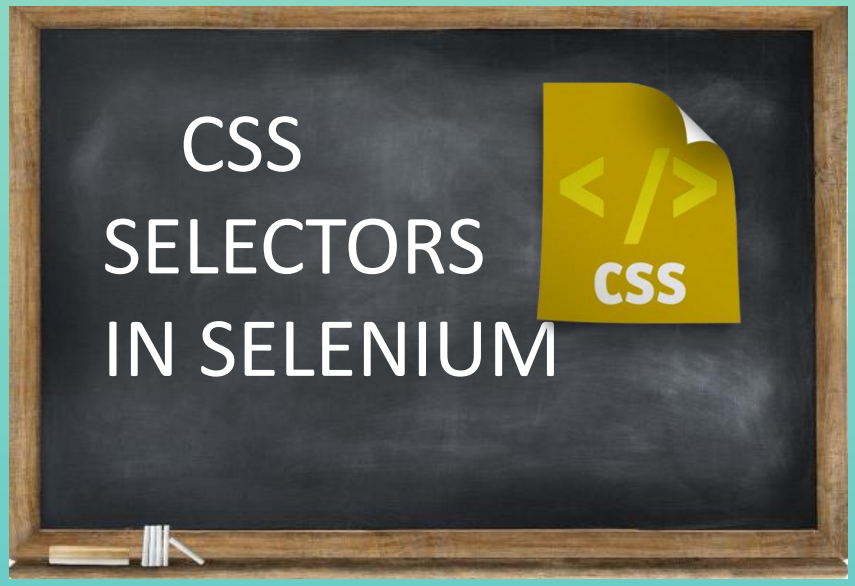
Selenium Locators

ID :	driver.findElement(By.Id("menu"));
Name :	driver.findElement(By.Name("home"));
Link Text :	driver.findElement(By.LinkText('Read on Wikipedia'));
Partial Link Text :	driver.findElementBy.PartialLinkText("Wikipedia"));
Class Name :	driver.findElement(ByClassName("container-top"));
Tag Name :	driver.findElement(By.TagName("a"));
CSS :	driver.findElement(By.CssSelector(."top-menu>li));
XPath :	driver.findElement(By.XPath("//"[@id=top-menu"));

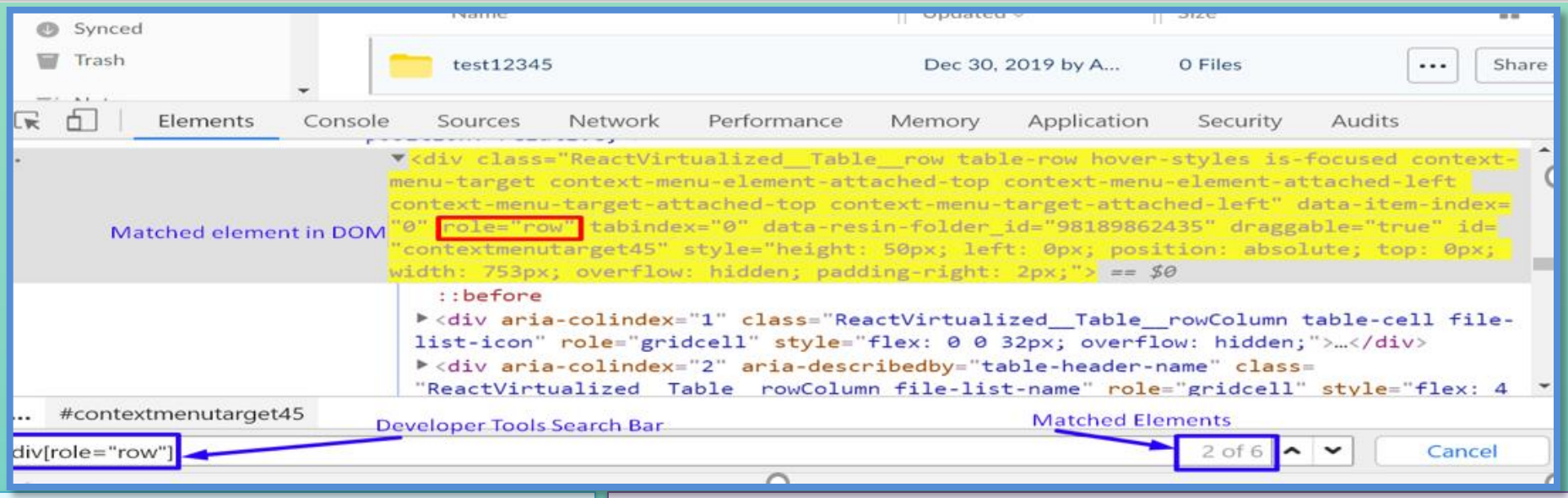
CSS Locators

- When we don't have an option to choose **Id, class or Name**, we should prefer using **CSS locators** as the best alternative
- **CSS Selector** is the combination of an **element selector** and a **selector value** which identifies the web element within a web page
- **CSS Selectors** are **string** representations of HTML tags, attributes, Id and Class

- <u>**Advantages**</u> of using CSS :-
  - **CSS** is a more **native approach** on all browsers
  - **CSS** is always the same **irrespective of browsers**
  - **CSS** is faster than **xpath**
  - **CSS** is more **readable** and easier to develop
  - **CSS** is used more often than **xpath**
  - Makes script more **dynamic and robust**



CSS with Element's Attribute(s)



- For evaluating and validating **CSS selectors** in **Chrome Developer tools**:
  - Press **Ctrl+F** to enable **DOM** searching in the panel
  - Type in → **CSS selector** to validate it
  - Corresponding matched element will be highlighted in **DOM**
  - Number of **matched elements** will be displayed on search bar

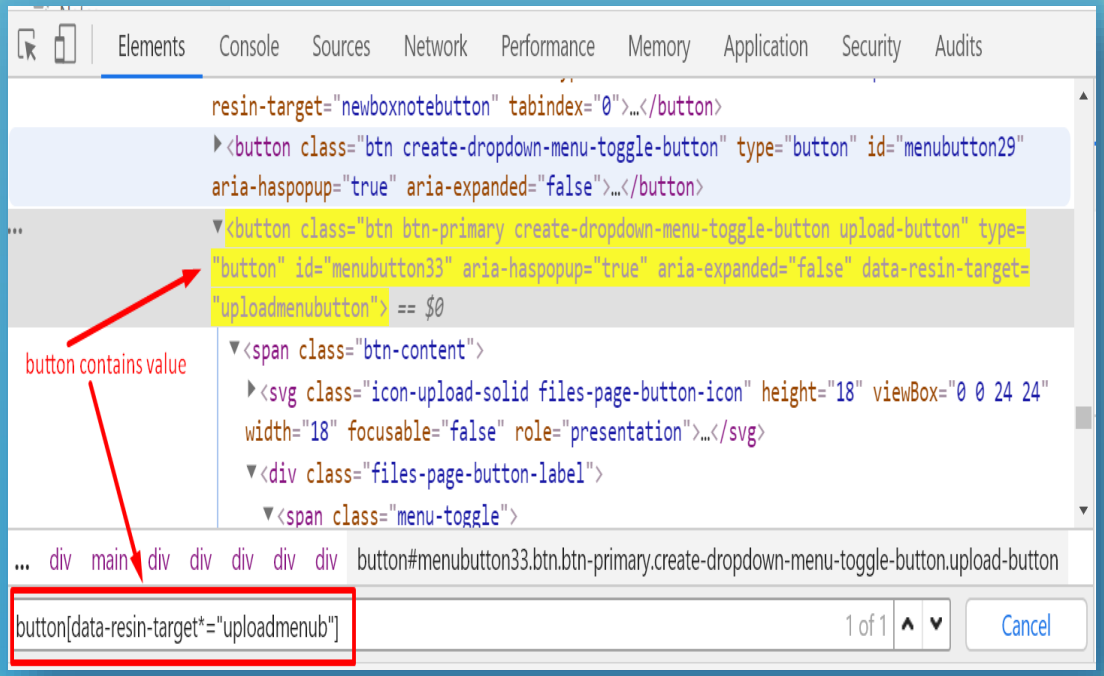
- Let's imagine we have a **HTML** tag with the following attributes **[id, class, name, type]:**
  <input type="text" id="fistname" name="first_name" class="myForm">
  - The generic way of locating elements by their attribute in **CSS Selectors** is:
    css =element_name[<attribute_name>="<value>"]
  - **CSS Selector** of above input tag becomes:
    css=input[name="first_name"]
  - **Element's CSS** with more than one attribute becomes:
    css=input[name='first_name'][id="fistname"]

More CSS Locating Techniques

- In **CSS,** there are **short forms** of **Id** and **Class** attribute that we can use to make it more readable.

- Let's imagine we have a tag with the following attributes [**id, class, name, type**]
  <input type="text" id="firstname" name="first_name" class="myForm">
  - In CSS, we can use (**#**) notation to select the id: css=input#firstname
  - We can also use the (**.**) notation to select the class: css=input.myForm

- **CSS For Element containing value:**
  Suppose, there is a **HTML** snippet:
  * *
  - To select the div element using "**sub-string**" or value it contains, in CSS we would use ***:**
    css = div[id*="_pattern_"]



Completing Login-Logout script

- **The <title> tag**:
  - It is required in all **HTML documents** and it defines the title of the document.
  - It is present in **<head> tag** which contains information related to the document.
  - You **CAN NOT** have more than **one <title>** element in an **HTML document**.
- In a **WebDriver** script, it's a good practice to verify **page title** whenever we navigate to a new page with method: *String getTitle();*

- **Get Text of WebElement**:
  - Getting text of **WebElement** is possible with **getText()** method which gets you the visible (i.e. not hidden by CSS) **innerText** of the element, including sub-elements.
  - Suppose there is a **HTML** snippet: ***<button id="login-submit" type="submit" class="btn btn-primary">Next</button>.***
  - Then code snippet: ***System.out.println(driver.findElement(By.id(" login-submit ")).getText());***
    This will print "**Next**"

```
String expectedTitle="Box | Login";
String actualTitle=driver.getTitle();
if(expectedTitle.equals(actualTitle)){
    System.out.println("successfully verified");
}else{
    System.out.println("Try Again");
}
```

gets current title on webpage

WebElement(s) for AccountMenu and logout in Box App

```
WebElement accountMenu = driver.findElement(By.cssSelector("button[data-resin-target='accountmenu']"));
accountMenu.click();

WebElement logoutLink = driver.findElement(By.cssSelector("a[data-resin-target='logout']"))
logoutLink.click();
```

Compound Class Name Error

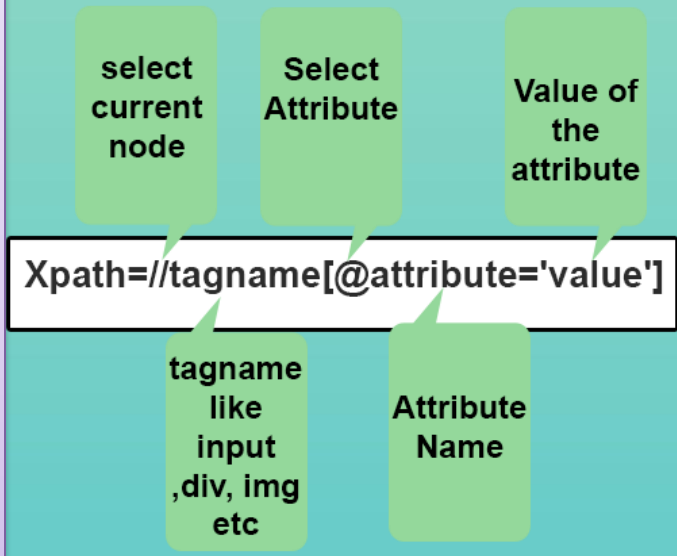
```
Exception in thread "main" org.openqa.selenium.InvalidSelectorException: invalid selector: Compound class names not permitted
  (Session info: chrome=79.0.3945.130)
  (Driver info: chromedriver=2.42.591088 (7b2b2dca23cca0862f674758c9a3933e685c27d5),platform=Windows NT 10.0.18362 x86_64) (WARNING: The serve
Command duration or timeout: 0 milliseconds
For documentation on this error, please visit: http://seleniumhq.org/exceptions/invalid_selector_exception.html
Build info: version: '3.14.0', revision: 'aacccce0', time: '2018-08-02T20:13:22.693Z'
System info: host: 'DREAMBOX', ip: '192.168.137.1', os.name: 'Windows 10', os.arch: 'amd64', os.version: '10.0', java.version: '1.8.0_191'
Driver info: org.openqa.selenium.chrome.ChromeDriver
Capabilities {acceptInsecureCerts: false, acceptSslCerts: false, applicationCacheEnabled: false, browserConnectionEnabled: false, browserName:
Session ID: 44a36193039592f3b46e37c3228d6bc7
```

- With HTML, you can assign **multiple values to a class attribute** separated with space but it will create confusion with **WebDriver** because the spaces between the class name confuses the **className()** method of **By class** as to which class it should look for.

- Suppose we have HTML snippet:
  <button id="login-submit" type="submit" class="btn btn-primary btn-full-width">
  - Then the following code snippet:
    driver.findElement(By.className("btn btn-primary btn-full-width"));
    This will throw "InvalidSelectorException".

XPath Locators

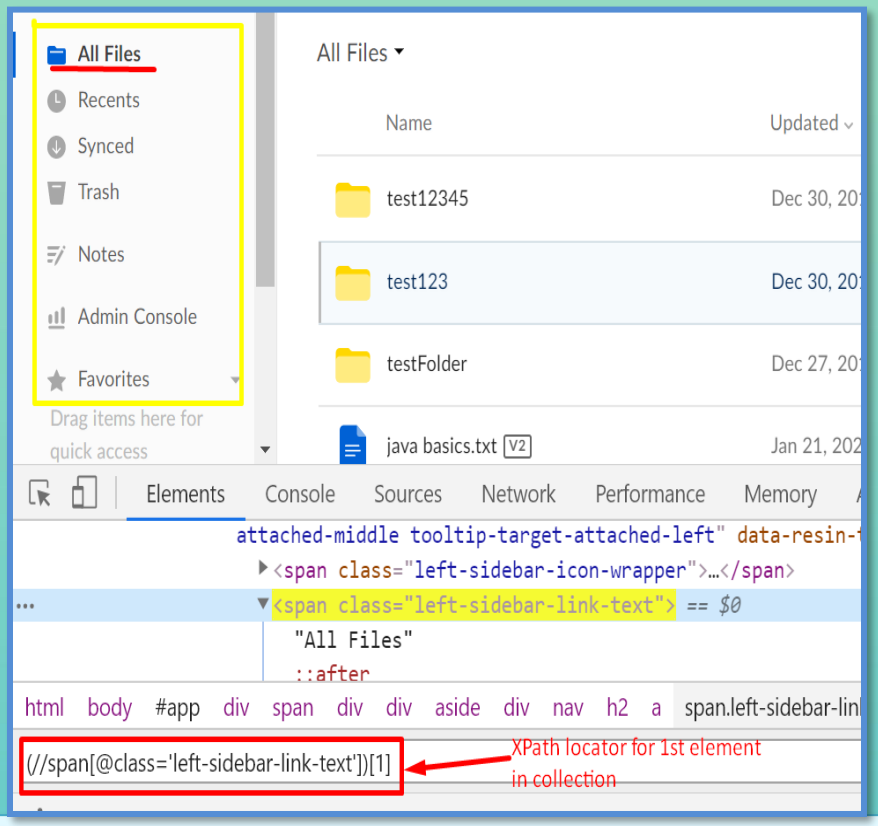
- **XPath is a syntax or language for finding any element on the web page using XML path expression**.
- It is the location of any element on a webpage using **HTML DOM** structure.
- There are two types of **XPath**:
  - **Absolute XPath**: It begins from **root node** and traverses to the element we need to find.
    - Writing **Absolute XPath** is very easy
    - Its very Long
    - It starts with **html** and then **single forward slash(/)**
    - It identifies the element very **fast**
    - This **XPath may break** if an extra element is added or an element is removed in between
      For Example -
      html/body/div[1]/section/div[1]/div/div/div/div[1]/div/div/div/div/div[3]/div[1]/div/h4[1]/b
  - **Relative XPath**:
    - It starts from any element which we want to use and **<u>not</u>** from the **root node**
    - It starts with **double forward slashes (*//*)**
    - The chances of failing **Relative XPath** are much less than **Absolute XPath**
      For Example: ***//div[@class='featured-box']***



More XPath Locating Techniques

- **Syntax for XPath:**
  XPath=//tagname[@attribute='value']
  Where
  - **// :** Select current node
  - **tagname:** Tagname of the particular node
  - **@:** Select attribute
  - **Attribute:** Attribute name of the node
  - **Value:** Value of the attribute

- **For the HTML snippet:**
  <a href="https://www.box.com/blog" class="header-marketing-link“ target="_blank“>
  XPath= //a[@class="header-marketing-link“]
- Both **XPath** and **CSS** can have attribute's value in **single quotes(‘)** as well as in **double quotes(“)**
  - Normally, we tend to use single quotes in **XPath** because in coding we have to use **escape sequence** while using double quotes



- **XPath with index**: When there are the **same XPath** locator for **multiple elements**, we can use index to locate a particular element
  - **XPath** indexing starts from **1**
  - Convert **Xpath** locator for multiple elements into a collection by surrounding it with **round brackets** and then using **index in square brackets.**

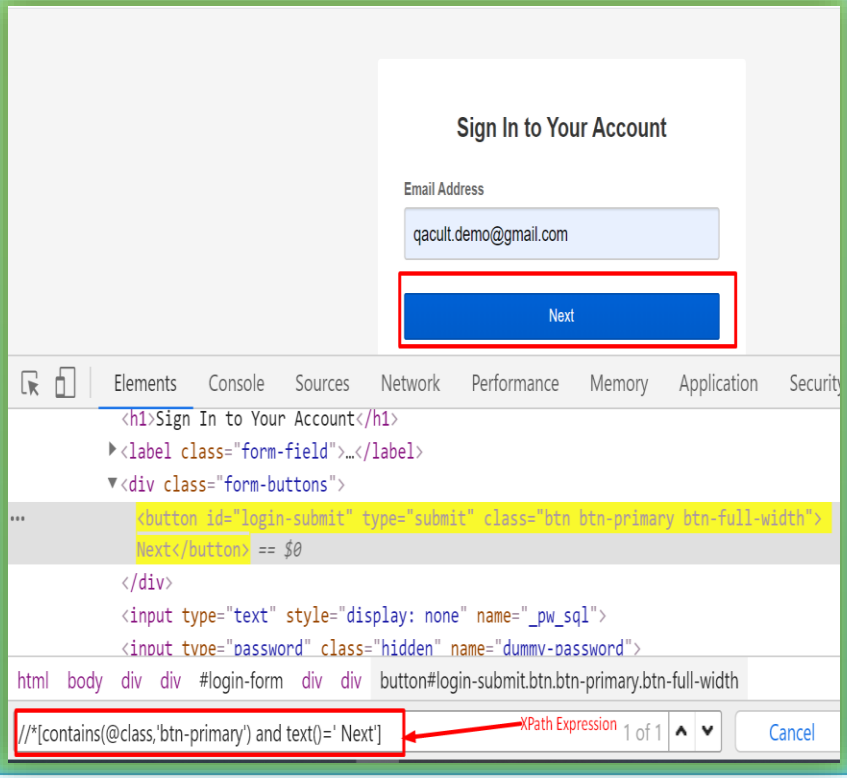
<u>For Example</u>: **XPath = //span[@class='left-sidebar-link-text']** is locator for **7 elements**
And **XPath = (//span[@class='left-sidebar-link-text'])[1]** is the locator for **1st element** in the **collection**

More XPath Locating Techniques

- Suppose we have **HTML** snippet:
  <button class="btn boxnote-button tooltip-target tooltip" type="button" aria-label="Write a quick note" data-resin-target="newboxnotebutton" tabindex="0“>Box Notes</button>

- *** in XPath**: We can use**(*)** in place of **tagname** in **XPath** which will select all the element nodes from the entire document.
  For example: **XPath=//*[@type="button"]** This will find all elements whose **attribute(type) value** is equal to **“button”**

- **Contains in XPath**: **Contains()** is a **method** used in **XPath expression** to validate **the part of the attribute value**. It is used when the value of any attribute changes **dynamically**.
  For example: **XPath=//button[contains(@class,'boxnote-button')]**



- **and & or in XPath**:
  - In **AND expression**, two conditions are used, both conditions should be true to find the element.
    For example: XPath=//button[@type="button" and @tabindex="0"]
  - In **OR expression**, two conditions are used and either of them should be true.
    For example: **XPath=//button[contains(@class,'boxnote-button') or @type="button"]**

- **Text in XPath**: Text is not the attribute of element but it is its property.
  For element’s text, the method used in **Xpath** is **text()**
  For example: **XPath=//button[text()=‘Box Notes’]**

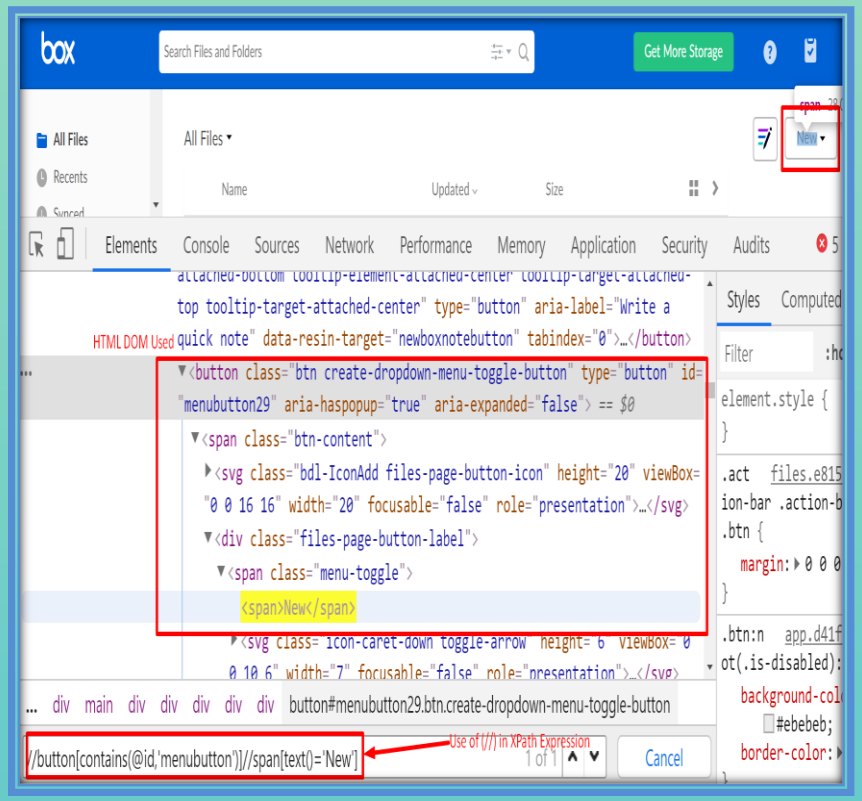
Single Slash(/) And Double Slash(//) in XPath

- Suppose we have a **HTML** snippet:

```
<div class="form-buttons">
        <button id="login" type="submit" class="btn-primary">Next</button>
        <div class="form-input">
                <input id="login-email" type="text" name="login" class="inp-width">
        </div>
</div>
```

- **Example with single-slash** which means moving to a child element at immediate level:
  Here we are finding XPath locator for button element.
  XPath=//div[@class='form-buttons']/button[@id='login']

- **Example with double-slash** which means moving to a child element at
  any level:
  Here we are finding the XPath locator of input element with reference to the first div element:
  XPath=//div[@class='form-buttons']//input[@id='login-email']



CSS Vs. XPath

Major Differences Between CSS and XPath

CSS	XPath
CSS can traverse **only down** the **HTML DOM**	XPath can traverse **up and down** the **HTML DOM**
CSS more **readable** (simpler)	somewhat **complex**
CSS is **faster** in browsers(especially in IE)	**slower** as compared to **CSS**
Latest **browsers optimize** the use of CSS Selectors	Not used in browsers **much**

Quiz

- Which two special methods are available in **By class** for locating **hyperlink**?
- How will you create **CSS** for an element with **attribute** and its **value**?
- Which special character will you use for **CSS** of an element with **attribute** and its **containing value**?
- Which **WebDriver** method will you use for getting the **title of webpage**?
- How will you get the text of **WebElement** in **WebDriver script**?
- How can you assign **multiple values** to a **class** attribute in **HTML** and what **error** will you get using (**By.className(String val)**) method in **WebDriver script**?
- How is **Absolute XPath** different from **Relative XPath**?
- How will you create **XPath** for element with **attribute** and its **value**?
- How will you create **XPath** for element **containing value** and having text?

Homework

- Create **WebDriver script** and **verify text**
  - Navigate to "**http://app.box.com**"
  - Click on "**Sign Up**" button
  - On new page, click on "**Individual Plans**" tab
  - Verify text "**Free**" is present in the "**Individual Plans**" tab.

- Create **WebDriver script** and **create folder**
  - Navigate to "**http://app.box.com**"
  - Enter username as **qacult.demo1@gmail.com** and click "**Next**"
  - Enter password as "testing@12345" and click on "**Log In**"
  - On **All Files page**, click on "**New**" button
  - Click on "**Folder**" option
  - In the **new pop up** window, create a new folder with text "**QATest**"
  - Click on "**Create**" button

HOMEWORK